Universidad Internacional del Ecuador · UIDE

Programación Orientada a Objetos (Go)

Tienda Go

Sistema de Gestión de E-commerce

Aprendizaje Autónomo 1

Etapa 1: Planeación del software (funciones y paquetes)

Estudiante:	Jhamir Rivera
Modalidad:	Individual
Unidad:	1 · Programación con Go (semanas 1 y 2)
Fecha de entrega:	15 de noviembre de 2026
Repositorio:	https://github.com/Jhamir2024/EcommerceGo

1. Objetivo del sistema

Permitir que una **tienda en línea pequeña** gestione su **catálogo de productos**, sus **clientes**, el **carrito de compras** y los **pedidos**, con control de stock y del total vendido, mediante un programa escrito en Go y organizado en paquetes.

Problema que resuelve: muchos emprendimientos llevan sus productos y ventas en cuadernos u hojas de cálculo. Es fácil vender algo que ya no hay en stock, olvidar un pedido o no saber cuánto se vendió. Tienda Go concentra esa información en un solo sistema y valida los datos antes de guardarlos.

Usuarios: el administrador de la tienda, que registra productos, clientes y pedidos.

2. Alcance

Lo que el sistema SÍ hace

- Registrar, consultar, cambiar de precio y eliminar productos, con control de stock.
- Registrar y consultar clientes, validando el formato del correo y que no se repita.
- Armar un carrito de compras por cliente, con subtotales y validación de stock.
- Confirmar pedidos: descuenta el stock y calcula el envío (gratis desde $50; si no, $3.50).
- Cancelar pedidos devolviendo el stock, y mostrar el total vendido.

Lo que el sistema NO hace (fuera del alcance)

- Pagos reales con tarjeta ni facturación electrónica.
- Envíos ni seguimiento de paquetes.
- Inicio de sesión con contraseña ni roles de usuario.
- Guardar los datos en una base de datos: en esta etapa todo vive en memoria mientras el programa corre.

3. Módulos del sistema

El sistema se divide en cuatro módulos de negocio y un módulo de apoyo. Cada módulo será un paquete de Go con una sola responsabilidad.

Módulo	Responsabilidad	Funcionalidades
Productos	Catálogo de la tienda y stock	Agregar, ver catálogo, buscar, cambiar precio, eliminar
Clientes	Personas que compran	Registrar, ver clientes
Carrito	Lo que el cliente quiere comprar	Agregar, ver con subtotal, quitar
Pedidos	Compras confirmadas	Confirmar, cancelar, ver pedidos y total vendido
Entrada (apoyo)	Leer datos del teclado	Leer texto, enteros y decimales

En total el sistema ofrece **13 funcionalidades** (ver sección 7), más de las 8 que pide el proyecto para la etapa final de servicios web.

4. Funciones por módulo

Las funciones con **mayúscula inicial** se exportan (se pueden usar desde otros paquetes). Las que empiezan con **minúscula** son auxiliares internas del paquete. Las funciones que pueden fallar devuelven un **error** como último valor.

4.1 Paquete productos

Función	Parámetros	Retorna	Qué hace
`AgregarProducto`	`nombre string, precio float64, unidades int`	`int, error`	Registra un producto y devuelve su ID
`ListarProductos`	—	`[]string`	Lista el catálogo ordenado por ID
`BuscarProducto`	`id int`	`string, float64, int, error`	Devuelve nombre, precio y stock
`ActualizarPrecio`	`id int, nuevoPrecio float64`	`error`	Cambia el precio de un producto
`EliminarProducto`	`id int`	`error`	Quita el producto del catálogo
`Existe`	`id int`	`bool`	Indica si el producto existe
`DescontarStock`	`id int, unidades int`	`error`	Resta stock al confirmar un pedido
`DevolverStock`	`id int, unidades int`	`error`	Suma stock al cancelar un pedido
`precioValido` (interna)	`precio float64`	`bool`	Valida que el precio sea mayor que 0

4.2 Paquete clientes

Función	Parámetros	Retorna	Qué hace
`RegistrarCliente`	`nombre string, correo string`	`int, error`	Registra un cliente y devuelve su ID
`BuscarCliente`	`id int`	`string, string, error`	Devuelve nombre y correo
`ListarClientes`	—	`[]string`	Lista los clientes ordenados por ID
`Existe`	`id int`	`bool`	Indica si el cliente existe
`correoValido` (interna)	`correo string`	`bool`	Revisa que el correo tenga @ y punto
`correoRegistrado` (interna)	`correo string`	`bool`	Evita correos repetidos

4.3 Paquete carrito

Función	Parámetros	Retorna	Qué hace
`AgregarAlCarrito`	`clienteID int, productoID int, unidades int`	`error`	Añade unidades validando cliente, producto y stock
`QuitarDelCarrito`	`clienteID int, productoID int`	`error`	Saca un producto del carrito
`VerCarrito`	`clienteID int`	`[]string`	Lista el carrito con subtotales
`CalcularTotal`	`clienteID int`	`float64, error`	Suma el carrito (sin envío)
`Contenido`	`clienteID int`	`map[int]int`	Entrega una copia del carrito al paquete pedidos
`VaciarCarrito`	`clienteID int`	—	Deja el carrito vacío
`idsOrdenados` (interna)	`clienteID int`	`[]int`	Ordena los productos del carrito

4.4 Paquete pedidos

Función	Parámetros	Retorna	Qué hace
`ConfirmarPedido`	`clienteID int`	`int, float64, error`	Compra el carrito: valida y descuenta stock, suma envío, devuelve ID y total
`CancelarPedido`	`pedidoID int`	`error`	Marca el pedido como cancelado y devuelve el stock
`ListarPedidos`	—	`[]string`	Lista los pedidos con cliente, total y estado
`TotalVendido`	—	`float64`	Suma los pedidos confirmados
`calcularEnvio (interna)`	`subtotal float64`	`float64`	Envío gratis desde $50; si no, $3.50

4.5 Paquete entrada (apoyo)

Función	Parámetros	Retorna	Qué hace
`LeerTexto`	`mensaje string`	`string`	Lee una línea completa (acepta espacios)
`LeerEntero`	`mensaje string`	`int, error`	Lee y convierte a número entero
`LeerDecimal`	`mensaje string`	`float64, error`	Lee un decimal con coma o punto

5. Diseño de paquetes

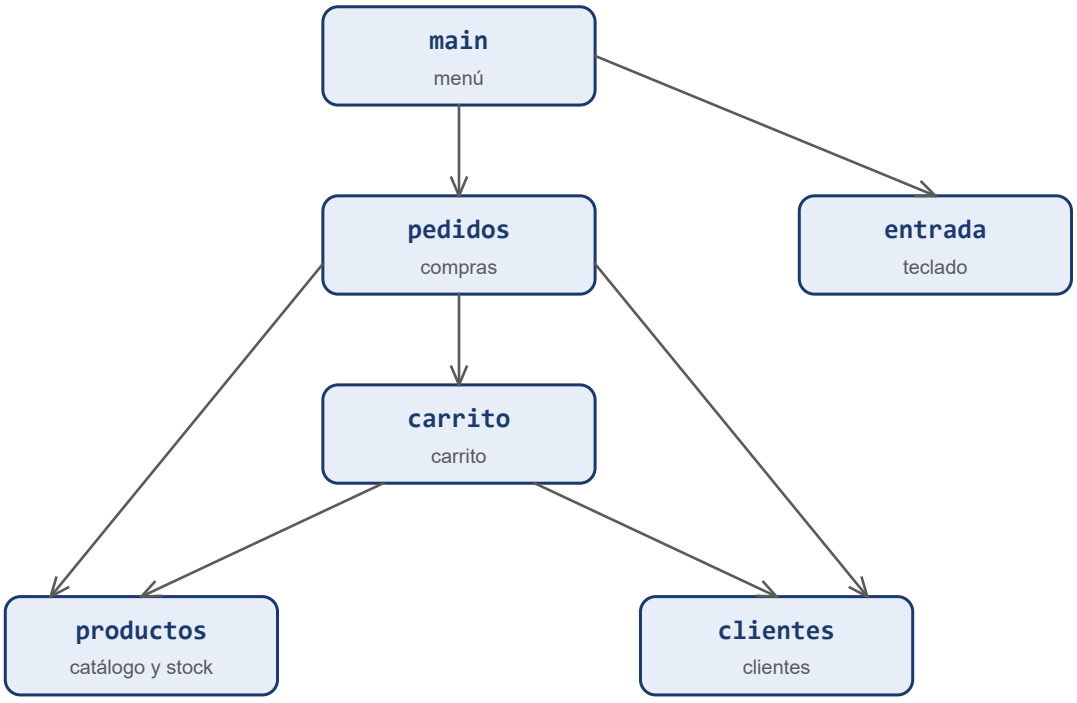
El módulo de Go se crea con **go mod init ecommercego**. Cada módulo del sistema es una carpeta con su paquete, y se importa con la ruta del módulo más la carpeta (por ejemplo, **ecommercego/productos**). El paquete **main** solo muestra el menú y pide datos; toda la lógica está en los paquetes.

5.1 Árbol del repositorio

```
EcommerceGo/                  módulo: ecommercego
├── go.mod
├── README.md
├── main.go                   menú de consola (paquete main)
├── productos/productos.go    package productos
├── clientes/clientes.go      package clientes
├── carrito/carrito.go        package carrito
├── pedidos/pedidos.go        package pedidos
├── entrada/entrada.go        package entrada
├── docs/                     este documento en PDF
└── practicas/semana01/       menú de la semana 1
```

5.2 Diagrama de dependencias

Una flecha A → B significa que el paquete A importa y usa funciones del paquete B.



main también importa productos, clientes y carrito para las opciones del menú (flechas omitidas para que el diagrama se lea mejor). No hay ciclos: ningún paquete de abajo importa a uno de arriba.

5.3 Qué importa y qué exporta cada paquete

Paquete	Importa	Exporta (públicas)	Internas
productos	errors, fmt	AgregarProducto, ListarProductos, BuscarProducto, ActualizarPrecio, EliminarProducto, Existe, DescontarStock, DevolverStock	precioValido, errNoExiste
clientes	errors, fmt, strings	RegistrarCliente, BuscarCliente, ListarClientes, Existe	correoValido, correoRegistrado
carrito	errors, fmt, sort, clientes, productos	AgregarAlCarrito, QuitarDelCarrito, VerCarrito, CalcularTotal, Contenido, VaciarCarrito	idsOrdenados
pedidos	errors, fmt, carrito, clientes, productos	ConfirmarPedido, CancelarPedido, ListarPedidos, TotalVendido	calcularEnvio
entrada	bufio, fmt, os, strconv, strings	LeerTexto, LeerEntero, LeerDecimal	—
main	fmt y los 5 paquetes propios	— (programa principal)	mostrarMenu y una función por opción

5.4 Paquetes de terceros

No se usan paquetes de terceros. Todo se resuelve con la biblioteca estándar de Go, que viene incluida y no necesita **go get**:

Paquete estándar	Para qué se usa
`fmt`	Mostrar texto y armar mensajes con formato
`errors`	Crear errores simples con errors.New
`strings`	Validar correos y limpiar el texto leído
`strconv`	Convertir el texto del teclado a int y float64
`bufio, os`	Leer líneas completas del teclado (con espacios)
`sort`	Ordenar los productos del carrito

Decisión: evitar dependencias externas mantiene el proyecto simple y garantiza que compile con **go build ./...** en cualquier computadora con Go. Si en etapas siguientes se agrega un paquete externo, se instalará con **go get**, quedará registrado en go.mod y se justificará en el README.

6. Cómo se guardan los datos en esta etapa

Como en la unidad 1 todavía no se usan structs, cada dato se guarda en un **map** cuya clave es el ID. Por ejemplo, en el paquete productos: **nombres[id]**, **precios[id]** y **stock[id]**. Estas variables están en minúscula, así que solo el propio paquete puede modificarlas; los demás paquetes deben usar sus funciones exportadas.

En el **Aprendizaje Autónomo 2** estos maps se reemplazarán por structs (**Producto**, **Cliente**, **Pedido**) con constructores y métodos, y las funciones de esta tabla pasarán a ser métodos.

7. Funcionalidades y servicios web previstos

Cada funcionalidad del menú se convertirá en un servicio web con JSON en la Evaluación en Contacto con el Docente:

#	Funcionalidad	Función principal	Servicio previsto
1	Agregar producto	`productos.AgregarProducto`	`POST /productos`
2	Ver catálogo	`productos.ListarProductos`	`GET /productos`
3	Buscar producto	`productos.BuscarProducto`	`GET /productos/{id}`
4	Cambiar precio	`productos.ActualizarPrecio`	`PUT /productos/{id}`
5	Eliminar producto	`productos.EliminarProducto`	`DELETE /productos/{id}`
6	Registrar cliente	`clientes.RegistrarCliente`	`POST /clientes`
7	Ver clientes	`clientes.ListarClientes`	`GET /clientes`
8	Agregar al carrito	`carrito.AgregarAlCarrito`	`POST /carrito/{cliente}`
9	Ver carrito	`carrito.VerCarrito`	`GET /carrito/{cliente}`
10	Quitar del carrito	`carrito.QuitarDelCarrito`	`DELETE /carrito/{cliente}/{producto}`
11	Confirmar pedido	`pedidos.ConfirmarPedido`	`POST /pedidos`
12	Ver pedidos y total	`pedidos.ListarPedidos`	`GET /pedidos`
13	Cancelar pedido	`pedidos.CancelarPedido`	`PUT /pedidos/{id}/cancelar`

8. Repositorio

Enlace: https://github.com/Jhamir2024/EcommerceGo

El repositorio contiene **go.mod**, la estructura de paquetes descrita en la sección 5, las funciones declaradas en la sección 4 y un **README.md** con la descripción del proyecto. Para comprobarlo:

```
go build ./...    # compila todos los paquetes sin errores
go run .          # abre el menú de la tienda
```